

Table of Contents

Objectives	3
1 Handling Missing Values.....	3
1.1 Detecting Missing Values	3
1.2 Handling Missing Values.....	4
2 Removing Duplicate Data	4
3 Data Type Conversion	4
3.1 Fixing Data Types	4
4 Feature Scaling.....	6
4.1 Normalization	6
4.2 Standardization	7
4.3 Feature Scaling During Testing Time	7
4.4 When to Use Normalization vs Standardization.....	8
5 Encoding	9
5.1 Label Encoding	9
5.2 One-Hot Encoding	10
5.3 Ordinal Encoding	11
5.4 Encoding at Testing Time	12

Objectives

After performing this lab, students shall be able to understand the following data science concepts and applications:

- ✓ Handling Missing/Duplicate Values, Outliers
- ✓ Data type conversions
- ✓ Encoding
- ✓ Feature Scaling

1 Handling Missing Values

Missing values occur when no data is recorded for certain entries in a dataset. This can happen due to data collection errors, incomplete surveys, or system failures.

City	State	Zip
Chicago	IL	<i>null</i>
San Antonio	TX	22259
<i>null</i>	TX	77343
<i>null</i>	TX	04927
San Diego	CA	16150
Philadelphia	PA	<i>null</i>
New York	<i>null</i>	25235
San Antonio	<i>null</i>	73868
Los Angeles	CA	24607
San Diego	CA	39289

Missing data can introduce bias, lead to incorrect conclusions, and cause errors in computations.

1.1 Detecting Missing Values

```
import pandas as pd

df = pd.read_csv("data.csv") # Load dataset
print(df.isnull().sum())    # Count missing values in each column
print(df.info())            # Check data types and missing values
```

1.2 Handling Missing Values

1.2.1 Remove rows/columns

Remove rows/columns with too many missing values:

```
df.dropna(inplace=True) # Remove rows with missing values
df.dropna(axis=1, inplace=True) # Remove columns with missing values
```

1.2.2 Imputation

Impute missing values (Mean/Median/Mode)

```
df['Age'].fillna(df['Age'].median(), inplace=True) # Fill missing
values with median
df['City'].fillna(df['City'].mode()[0], inplace=True) # Fill missing
values with mode
```

2 Removing Duplicate Data

Duplicates can inflate dataset size and lead to incorrect insights.

```
df.duplicated().sum() # Check for duplicate rows
df = df.drop_duplicates() # Remove duplicate rows
```

3 Data Type Conversion

Data type conversion is the process of changing a data column's type (like from text to number or string to date). In data analysis or machine learning, having the correct data types is crucial. For example:

- If prices are stored as strings ("123.45"), you can't calculate totals or averages correctly.
- If dates are stored as text ("2023-01-01"), you can't sort them or calculate durations properly.

3.1 Fixing Data Types

3.1.1 `pd.to_numeric()` → Convert to numeric (int or float)

```
df['Price'] = pd.to_numeric(df['Price'], errors='coerce')
```

Convert strings like "123.45" or "1,000" to numeric values.

Options for errors:

Value	Description
'raise' (default)	Raise an error if conversion fails.
'coerce'	Invalid parsing will be set as NaN.
'ignore'	Return original data if parsing fails.

3.1.2 `pd.to_datetime()` → Convert to datetime

```
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
```

Supports various date formats automatically.

3.1.3 `.astype()` → Convert to specific type

```
df['Quantity'] = df['Quantity'].astype(int)
```

Explicitly cast a column to a specific type (int, float, str, etc.)

Can be used for conversions like:

- int → integer
- float → floating point
- str → string
- category → efficient for categorical data

Will raise an error if conversion fails. So make sure the data is clean or use `errors='coerce'` first.

3.1.4 `infer_objects()` → Let Pandas guess better data types

```
df = df.infer_objects()
```

If your DataFrame has object types, this tries to convert them to more specific types like int, float, or datetime based on content.

3.1.5 `convert_dtypes()` → Automatically assign best data types

```
df = df.convert_dtypes()
```

Automatically converts all columns to the best possible data type (including new nullable types like Int64, string, boolean, etc.)

Especially useful when loading raw data from CSV/Excel files.

4 Feature Scaling

Feature scaling is a data preprocessing technique used to bring numerical features onto a similar scale. It is especially important for machine learning algorithms that rely on distance, gradients, or variance, such as k-NN, SVM, linear regression, and neural networks. Without feature scaling, variables with larger ranges can dominate the learning process and lead to poor model performance.

Feature scaling:

- Improves model convergence speed
- Prevents bias toward large-magnitude features
- Helps distance-based algorithms work correctly
- Stabilizes gradient descent optimization

Tree-based models (e.g., Decision Trees, Random Forests) usually do not require feature scaling.

4.1 Normalization

Normalization rescales data to a fixed range, typically between 0 and 1. It preserves the shape of the original distribution but compresses the scale.

Normalization is useful when:

- Data does not follow a normal distribution (skewness...)
- You are using distance-based algorithms (k-NN, k-means)
- Feature bounds are known and meaningful

It is calculated using:

$$X_{\text{new}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Sample data
X = np.array([[10], [20], [30], [40], [50]])
```

```
# Normalization
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
print(X_normalized)
```

4.2 Standardization

Standardization transforms data to have:

- Mean = 0
- Standard deviation = 1

It does not restrict values to a fixed range and works well when data follows a normal distribution.

It is calculated using:

$$X_{\text{new}} = (X - \text{mean}) / \text{Std}$$

Standardization is preferred when:

- Data contains outliers
- Features follow a Gaussian distribution
- Using algorithms like linear regression, logistic regression, SVM, or PCA

```
from sklearn.preprocessing import StandardScaler

# Sample data
X = np.array([[10], [20], [30], [40], [50]])

# Standardization
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)
print(X_standardized)
```

4.3 Feature Scaling During Testing Time

During testing (or prediction) time, feature scaling does not change its formula, but how it is applied changes.

- The scaler is fit only on the training data
- During testing, the scaler is reused, not refit
- The same parameters learned from training are applied to test data

This ensures fairness and prevents data leakage.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# Sample dataset
X = np.array([[10], [20], [30], [40], [50]])

# Train-test split
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

# Fit scaler on training data only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Apply same scaler to test data
X_test_scaled = scaler.transform(X_test)

print("Train scaled:\n", X_train_scaled)
print("Test scaled:\n", X_test_scaled)
```

4.4 When to Use Normalization vs Standardization

The choice between normalization and standardization depends mainly on the distribution of the data and the presence of skewness. If a feature is approximately normally distributed (symmetric and bell-shaped), standardization is preferred because it centers the data around zero and scales it based on variance. If the data is not normally distributed or has clear bounds, normalization is often more suitable, especially for distance-based algorithms.

Skewness is a key indicator in this decision. Skewness measures the asymmetry of a distribution and can be visually checked using a histogram or numerically computed using the `.skew()` function. A skewness value close to 0 indicates a symmetric distribution. In practice, if skewness lies roughly between -0.5 and $+0.5$, the data can be treated as approximately normal and standardization is recommended. If skewness is greater than $+1$ or less than -1 , the distribution is highly skewed, and normalization is generally preferred unless the data is transformed first (e.g., log transformation).

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from scipy.stats import skew

# Sample data
data = np.array([10, 12, 15, 18, 20, 22, 25, 100])

# Histogram
plt.hist(data, bins=10)
plt.title("Histogram of Feature")
plt.show()

# Skewness value
skew_value = skew(data)
print("Skewness:", skew_value)
```

Rule of thumb:

- Skewness between -0.5 and $+0.5$: safe to use standardization
- Skewness > 1 or < -1 : prefer normalization

5 Encoding

Encoding is a feature engineering technique used to convert categorical variables into numerical form. Since most machine learning algorithms operate only on numerical data, categorical features such as colors, countries, or severity levels must be encoded before model training.

The choice of encoding method depends on whether the categorical data has no order (nominal) or a meaningful order (ordinal).

5.1 Label Encoding

Label Encoding converts each category into a unique integer value. The entire categorical column is replaced by these numeric labels.

For example:

- Red: 0
- Green: 1
- Blue: 2

Label encoding is suitable when:

- The data is ordinal (has a natural order)
- You are using tree-based models (Decision Trees, Random Forests)

- The numeric ordering will not mislead the model

Using label encoding on nominal data can be dangerous, as models may incorrectly assume a mathematical relationship between categories.



```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Categorical data
colors = ['Red', 'Blue', 'Green', 'Blue', 'Red']
df = pd.DataFrame({'Color': colors})

label_encoder = LabelEncoder()
df['Color_encoded'] = label_encoder.fit_transform(df['Color'])
print(df)
```

5.2 One-Hot Encoding

One-Hot Encoding converts a categorical feature into multiple binary columns, one for each unique category. A value of 1 indicates the presence of that category, while 0 indicates absence.

For example, a Country feature with values USA, Canada, and India becomes three separate columns.

One-hot encoding is best when:

- The data is nominal (no inherent order)
- The number of unique categories is small
- You are using models like logistic regression, KNN, or neural networks

Its main drawback is increased dimensionality, which can lead to sparse data.

Color	
Red	
Red	
Yellow	
Green	
Yellow	



Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1

```
import pandas as pd

countries = ['USA', 'Canada', 'India', 'USA', 'Canada']
df = pd.DataFrame({'Country': countries})

# One-Hot Encoding
one_hot = pd.get_dummies(df['Country'])
print(one_hot)
```

5.3 Ordinal Encoding

Ordinal Encoding assigns numerical values to categories based on a predefined order. Unlike label encoding, the order is explicitly defined by the user, making it safer and more meaningful for ordinal data.

For example:

- Low: 1
- Medium: 2
- High: 3

Ordinal encoding should be used when:

- Categories have a clear and meaningful ranking
- The order itself carries information
- You want to preserve relative importance between categories

Original Encoding	Ordinal Encoding
Poor	1
Good	2
Very Good	3
Excellent	4

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# Ordinal data
severity = ['Low', 'Medium', 'High', 'Medium', 'Low']
df = pd.DataFrame({'Severity': severity})

encoder = OrdinalEncoder(categories=[['Low', 'Medium', 'High']])
df['Severity_encoded'] = encoder.fit_transform(df[['Severity']])
print(df)
```

5.4 Encoding at Testing Time

During testing or prediction time, encoding methods must not be refit. Just like feature scaling, encoders are learned from the training data and reused unchanged on test data. This ensures consistency and prevents data leakage.

The key rule is:

fit on training data, transform on test data.

So instead of using `.fit_transform`, you only need to use `.transform`. Consider the example below:

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# Training data
train = pd.Series(['Low', 'Medium', 'High'])

# Testing data
test = pd.Series(['Medium', 'Low'])
```

```
encoder = LabelEncoder()

# Fit on training data
encoder.fit_transform(train)

# Transform test data
test_encoded = encoder.transform(test)
print(test_encoded)
```